

面向动态柔性作业车间重调度的强化学习引导逻辑 Benders 展开求解框架

博士论文开题学术方案

作者：XXX

导师：XXX

2026 年 5 月 15 日

摘要

动态柔性作业车间重调度要求在机器故障、新订单到达、加工时间扰动等事件发生后，快速生成兼顾效率与稳定性的修复方案。传统精确优化方法具有严格的可行性建模能力，但在大规模动态场景下常受限于搜索空间膨胀；学习型调度方法具备快速推断优势，却往往缺乏可解释的约束推理结构和可验证的最优性逻辑。本课题拟提出一种强化学习引导的逻辑 Benders 分解展开框架，将动态柔性作业车间修复问题建模为带稳定性正则的组合优化问题，并将逻辑 Benders 分解的迭代过程视为可展开的决策轨迹。框架中，主问题负责机器分配、冻结/解冻操作选择与修复区域规划，子问题通过约束规划或小规模混合整数规划完成局部精确排程；强化学习策略不直接生成完整排产方案，而是学习每轮分解中的修复区域、割平面优先级与子问题预算控制。预期该方法能够在保持数学分解结构与可解释性的同时，显著改善动态重调度的 anytime 性能、稳定性与跨规模泛化能力。

1 研究背景与动机

柔性作业车间调度问题 (Flexible Job-Shop Scheduling Problem, FJSP) 是制造系统、半导体生产、装备维修、智能仓储和离散制造中的核心组合优化问题。与经典作业车间调度问题相比，FJSP 同时包含操作顺序约束和机器选择决策：每个工件由若干具有前后依赖关系的操作组成，每个操作可由一组候选机器之一加工，不同机器上的加工时间可能不同。该问题同时涉及机器分配、同机排序、开始时间安排和目标函数优化，具有典型的 NP-hard 结构 [1, 2]。

在真实制造环境中，排产方案很少处于静态状态。机器故障、急单插入、工件到达时间变化、加工时间延迟、设备维护窗口变更等扰动会使原计划部分失效。此时，调度系统的目标并不是完全重新求解一个全局最优排产，而是在有限时间内生成一个可行且稳定

的修复方案。因此，动态柔性作业车间重调度 (Dynamic Flexible Job-Shop Rescheduling, DFJSP-Repair) 需要同时优化：

- **生产效率**：如最大完工时间、总拖期、加权拖期、机器负载均衡；
- **计划稳定性**：如开始时间偏移、机器重分配次数、关键工件顺序改变；
- **响应速度**：扰动发生后在短时间内给出高质量可行修复方案；
- **约束可验证性**：修复方案必须满足工艺前后约束、机器容量约束和扰动后不可用窗口约束。

现有调度研究大致形成了两条路线。第一类是基于数学规划、约束规划、逻辑 Benders 分解、分支定界等方法的精确或近精确算法，其优势在于建模严谨、可解释性强、可给出下界或可行性证据，但在动态、大规模、多目标场景中往往受限于计算时间 [5, 6]。第二类是基于图神经网络、强化学习、模仿学习或神经组合优化的学习型方法，其优势在于推断速度快、能够从数据中学习启发式规则，但多数方法将调度过程简化为逐步派工决策，难以处理复杂重调度中的稳定性约束、局部修复需求和可验证的约束推理 [9, 10]。

本课题的核心动机是：**与其让神经网络直接替代调度求解器，不如让学习方法控制一个结构化、可解释、可验证的分解求解过程**。逻辑 Benders 分解 (Logic-Based Benders Decomposition, LBBD) 天然具有“主问题-子问题-逻辑割-再优化”的迭代结构。若将每一轮分解视为一个可展开的求解层，则可以将其建模为强化学习环境：状态是当前排产方案、扰动信息、下界、割池和历史修复效果；动作是选择修复区域、割平面优先级、子问题时间预算或搜索模式；奖励来自目标值改善、gap 缩小、可行性恢复和计算时间节约。由此形成一种介于精确优化与学习启发式之间的新型算法范式，即**强化学习引导的 LBBD 展开求解器**。

2 相关工作与前人研究（指出现有工作的不足，明确本文的差异化创新）

2.1 柔性作业车间调度与动态重调度

FJSP 的经典建模通常包括机器选择变量、操作开始时间变量和同机排序变量，可通过混合整数规划、析取图、约束规划或元启发式算法求解 [1, 2]。动态重调度进一步引入扰动事件和计划稳定性约束，要求在原始排产方案附近进行局部调整，而不是完全重排。已有研究常采用滚动时域、事件驱动重调度、局部搜索、遗传算法、禁忌搜索、模拟退火或自适应大邻域搜索等方法 [3, 4]。这些方法在工程中较为实用，但依赖大量人工设计的邻域、参数和接受准则，且在复杂约束下缺少统一的可验证最优性逻辑。

本课题与上述方法的主要差异在于：本文不将重调度视为单纯的元启发式搜索，而是将其嵌入 LBBD 的分解框架中；同时，强化学习并不直接决定操作序列，而是学习如

何控制分解求解过程中的修复区域和搜索资源，从而在保持可解释结构的前提下提高搜索效率。

2.2 数学规划、约束规划与逻辑 Benders 分解

混合整数规划 (Mixed-Integer Programming, MIP) 能够统一表达机器分配、排序和时间约束，但对于大规模 FJSP 往往面临大 M 约束松弛弱、二元排序变量数量庞大和分支树增长过快等问题。约束规划 (Constraint Programming, CP) 在调度中具有显著优势，尤其适合表达 interval variable、optional interval 和 no-overlap 约束 [7, 8]。但单纯 CP 在需要提供全局下界、处理复杂多目标分配结构或嵌入学习控制时仍存在局限。

逻辑 Benders 分解通过将原问题拆分为主问题和子问题，使主问题处理高层离散决策，子问题验证底层可行性和排程质量，并以逻辑割形式反馈信息 [5]。对 FJSP 而言，一种自然分解是：主问题决定操作-机器分配、冻结/解冻变量和粗粒度修复计划；子问题在给定高层决策后求解具体排程。LBBD 的优势是结构清晰、可解释、可与 CP/MIP 混合，但传统 LBBD 的性能依赖割平面设计、子问题求解预算、修复区域选择和搜索顺序。现有研究多以人工规则或固定策略处理这些控制决策，缺乏从实例分布和历史求解轨迹中学习的机制。

本文的差异化创新在于：将 LBBD 的迭代过程显式建模为一个可展开的序列决策过程，并用强化学习学习那些传统算法中难以手工优化的控制决策。该设计避免了直接神经化 CP/MIP 求解器内部复杂机制，同时保留了分解算法的逻辑结构。

2.3 图神经网络与强化学习调度方法

近年来，图神经网络与强化学习被广泛用于 TSP、VRP、JSSP、MaxCut、图着色等组合优化问题 [10, 11, 12]。在 JSSP 中，常见做法是将调度状态表示为析取图或操作图，用 GNN 编码操作、机器和前后关系，再通过策略网络选择下一个可调度操作 [9]。该路线本质上是学习派工规则，在静态小中规模问题上具有一定效果。

然而，从资深审稿人的角度看，这类工作存在三点不足。第一，许多方法将复杂调度简化为逐步构造，忽略了动态扰动后“局部修复而非重排”的核心需求。第二，端到端策略通常缺少可行性证书、下界估计和结构化约束反馈，难以与工业级优化器竞争。第三，若仅替换 GNN 编码器或 RL 算法，创新性容易局限于工程增量。

本文不沿用“GNN + RL 直接派工”的范式，而是提出“GNN/RL 控制 LBBD 展开”的范式。GNN 主要用于编码当前排产图、扰动图和割池图；RL 主要用于选择修复区域、割平面优先级和求解预算；可行性验证和局部排程仍由 CP/MIP 子问题完成。这一差异使本文在算法机制上更接近可解释的学习增强优化，而非黑箱神经调度器。

2.4 学习增强的分解算法与算法展开

算法展开 (algorithm unrolling) 通过将迭代优化算法的若干步展开为可学习网络层, 在稀疏恢复、图像重建、凸优化和运筹优化中具有广泛应用 [13]. 在组合优化领域, 学习方法也被用于分支变量选择、割平面选择、列生成定价、局部搜索控制和大邻域搜索策略选择 [14, 15, 16]. 这些研究表明, 学习算法最有价值的位置往往不是替代整个求解器, 而是提升求解器内部关键控制决策。

本文与已有学习增强分解算法的区别在于: 本文聚焦动态 FJSP 修复问题, 并将 LBBD 的“主问题-子问题-逻辑割”循环作为展开单元。强化学习策略作用于重调度中特有的修复区域选择、稳定性约束管理和 anytime 求解预算控制, 而不仅是一般意义上的割选择或分支变量选择。因此, 本文预期能够形成一个针对动态排产修复任务的系统性方法框架。

3 研究目的与研究意义

本课题的总体目标是研究一种面向动态柔性作业车间重调度的学习增强分解优化方法, 构建既具备数学优化可解释性、又具备学习型搜索适应性的 unrolled solver。具体目标包括:

1. 建立带稳定性正则的动态 FJSP 修复优化模型, 统一描述机器故障、新订单插入、加工时间变化等扰动下的局部重调度需求;
2. 设计适用于该模型的 LBBD 分解框架, 明确主问题、子问题、逻辑割、下界更新和可行性恢复机制;
3. 将 LBBD 迭代过程形式化为马尔可夫决策过程, 使每一轮分解成为可展开、可学习、可评估的求解步骤;
4. 设计基于异构图神经网络的状态编码器和强化学习控制策略, 学习修复区域选择、割平面排序和子问题预算分配;
5. 在标准 FJSP/JSSP 基准和合成动态扰动场景上验证方法相对于传统 LBBD、CP-SAT、局部搜索、派工规则和神经调度方法的性能优势;
6. 分析该框架在可解释性、泛化性、anytime 性能和工程可扩展性方面的潜力与局限。

研究意义体现在三个层面。理论层面, 本课题尝试将逻辑 Benders 分解与强化学习控制统一到展开优化框架中, 为“学习增强分解算法”提供新的建模样式。方法层面, 本课题避免纯神经组合优化缺少可行性验证的弱点, 也避免精确算法在动态场景中搜索控制僵化的问题。应用层面, 动态重调度是智能制造系统中的高频需求, 具有明确的工业价值; 若能在秒级或分钟级预算内提供高质量稳定修复方案, 将显著提升生产系统韧性。

4 研究内容与核心方法（详细描述算法框架、理论推导或模型设计的关键思路，突出新颖性）

4.1 动态 FJSP 修复问题建模

设工件集合为 $\mathcal{J} = \{1, \dots, J\}$ ，机器集合为 $\mathcal{M} = \{1, \dots, M\}$ ，操作集合为 $\mathcal{O}$ 。每个操作 $o \in \mathcal{O}$ 属于某个工件，并具有前驱集合 $\text{Pred}(o)$ 。操作 o 可由候选机器集合 $\mathcal{M}_o \subseteq \mathcal{M}$ 中的一台机器加工；若分配到机器 m ，加工时间为 p_{om} 。原始排产方案记为

$$x^0 = \{s_o^0, c_o^0, m_o^0, \pi_m^0\}_{o \in \mathcal{O}, m \in \mathcal{M}}, \quad (1)$$

其中 s_o^0 和 c_o^0 分别为开始与完成时间， m_o^0 为原机器分配， π_m^0 为机器 m 上的操作顺序。

扰动事件记为 $d \in \mathcal{D}$ ，可包含机器不可用窗口 $[a_m, b_m]$ 、新到达工件集合 $\mathcal{J}^{new}$ 、加工时间变化 Δp_{om} 或急单 due date 变化。重调度的目标是生成新方案

$$x = \{s_o, c_o, m_o, \pi_m\}, \quad (2)$$

使其满足扰动后的可行性约束，并在效率与稳定性之间折中。本文采用如下 proximal rescheduling objective:

$$\begin{aligned} \min_x \quad \text{Obj}(x) = & \alpha C_{\max}(x) + \beta \sum_{j \in \mathcal{J}} T_j(x) + \gamma \sum_{o \in \mathcal{O}} |s_o - s_o^0| \\ & + \eta \sum_{o \in \mathcal{O}} \mathbb{I}[m_o \neq m_o^0] + \kappa \sum_{m \in \mathcal{M}} d_{\text{seq}}(\pi_m, \pi_m^0), \end{aligned} \quad (3)$$

其中 $T_j = \max\{0, C_j - D_j\}$ 为工件拖期， d_{seq} 衡量机器序列扰动。约束包括：

$$\sum_{m \in \mathcal{M}_o} y_{om} = 1, \quad \forall o \in \mathcal{O}, \quad (4)$$

$$c_o = s_o + \sum_{m \in \mathcal{M}_o} p_{om} y_{om}, \quad \forall o \in \mathcal{O}, \quad (5)$$

$$s_v \geq c_o, \quad \forall o \in \text{Pred}(v), \quad (6)$$

$$[s_o, c_o) \cap [s_v, c_v) = \emptyset, \quad \forall o \neq v, m_o = m_v, \quad (7)$$

$$[s_o, c_o) \cap [a_m, b_m) = \emptyset, \quad \forall o : m_o = m. \quad (8)$$

对已经开始或不可中断的操作，可加入固定约束 $s_o = s_o^0, m_o = m_o^0$ 。公式(3)–(8)构成本文的基础优化模型。

4.2 LBBD 分解框架

本文将问题拆分为主问题 MP 和子问题 SP。主问题处理高层离散决策，子问题处理给定高层决策下的精确时间排程。

4.2.1 主问题设计

第 k 轮主问题包含以下变量：机器分配变量 y_{om} 、操作解冻变量 u_o 、修复区域变量 $R_k = \{o : u_o = 1\}$ 、目标下界变量 θ 。主问题可写为：

$$\begin{aligned}
\min_{y, u, \theta} \quad & \theta + \lambda_u \sum_{o \in \mathcal{O}} u_o + \lambda_r \sum_{o \in \mathcal{O}} \sum_{m \in \mathcal{M}_o} \mathbb{I}[m \neq m_o^0] y_{om} \\
\text{s.t.} \quad & \sum_{m \in \mathcal{M}_o} y_{om} = 1, & \forall o \in \mathcal{O}, \\
& y_{om_o^0} = 1, & \forall o : u_o = 0, \\
& u_o = 0, & \forall o \in \mathcal{O}^{fixed}, \\
& (y, u, \theta) \in \mathcal{C}_k,
\end{aligned} \tag{9}$$

其中 $\mathcal{C}_k$ 为前 $k-1$ 轮积累的逻辑割集合。主问题并不完整决定所有开始时间，而是输出候选机器分配和修复区域。

4.2.2 子问题设计

给定主问题输出 (y^k, u^k) ，子问题在修复区域 $R_k = \{o : u_o^k = 1\}$ 内重新安排操作，在区域外保持原排产或仅允许有限松弛。子问题可由 CP-SAT、CP Optimizer 或小规模 MIP 求解：

$$\begin{aligned}
\min_{s, c, \pi} \quad & \text{Obj}(x) \\
\text{s.t.} \quad & \text{precedence constraints,} \\
& \text{machine no-overlap constraints,} \\
& \text{disturbance availability constraints,} \\
& m_o = \arg \max_{m \in \mathcal{M}_o} y_{om}^k, \\
& (s_o, m_o, \pi_o) = (s_o^0, m_o^0, \pi_o^0), \quad \forall o \notin R_k \text{ subject to repair policy.}
\end{aligned} \tag{10}$$

若子问题不可行，则返回可行性割；若可行，则更新 incumbent 并返回最优性割或 critical-path cut。

4.2.3 逻辑割设计

本文拟设计四类核心割。

No-good assignment cut. 若某一分配组合导致不可行，设不可行核心为 $Q_k \subseteq \{(o, m) : y_{om}^k = 1\}$ ，则加入：

$$\sum_{(o, m) \in Q_k} y_{om} \leq |Q_k| - 1. \tag{11}$$

该割排除导致冲突的同类分配组合。

Repair-region feasibility cut. 若当前修复区域过小导致无法恢复可行性, 设冲突操作集合为 B_k , 则加入:

$$\sum_{o \in B_k} u_o \geq q_k, \quad (12)$$

其中 q_k 可由冲突强度、机器故障窗口和前后约束传播结果决定。

Critical-path optimality cut. 若子问题可行但被关键路径 P_k 限制, 则加入下界割:

$$\theta \geq \sum_{o \in P_k} \sum_{m \in \mathcal{M}_o} p_{om} y_{om} + \ell(P_k), \quad (13)$$

其中 $\ell(P_k)$ 表示由释放时间、等待时间、机器不可用窗口或序列关系诱导的附加下界。

Stability-aware cut. 若某候选修复方案导致过大扰动, 则加入稳定性限制:

$$\sum_{o \in R_k} |s_o - s_o^0| + \rho \sum_{o \in R_k} \mathbb{I}[m_o \neq m_o^0] \leq \Delta_k. \quad (14)$$

该类割体现动态重调度与静态排产的本质区别。

4.3 将 LBBD 展开为马尔可夫决策过程

传统 LBBD 的性能依赖人工策略, 例如优先修复哪些机器块、何时扩大邻域、哪些割优先加入、子问题给多少时间预算。本文将这些策略学习化。定义一个 episode 为一次扰动后的完整重调度过程, LBBD 的第 k 轮为 MDP 的第 k 个时间步。

状态空间. 状态 s_k 包含:

- 当前 incumbent 排产方案 x_k^{best} ;
- 扰动事件 d 及受影响操作集合;
- 当前主问题下界 LB_k 、上界 UB_k 和 gap;
- 割池 C_k 的类型、年龄、历史有效性;
- 最近若干轮修复区域、目标改善和子问题求解时间;
- 操作 slack、critical path 标记、机器负载、due-date pressure 等图特征。

动作空间. 第一阶段采用分层动作:

$$a_k = (a_k^{anchor}, a_k^{type}, a_k^{radius}, a_k^{budget}), \quad (15)$$

其中 a_k^{anchor} 选择一个锚点操作或机器块, a_k^{type} 选择邻域类型 (同工件、同机器、关键路径、时间窗口、扰动传播邻域), a_k^{radius} 决定邻域大小, a_k^{budget} 决定子问题时间预算。动作经解码函数 Expand 生成修复区域:

$$R_k = \text{Expand}(a_k^{anchor}, a_k^{type}, a_k^{radius}; G_k). \quad (16)$$

后续阶段将扩展到割选择动作:

$$a_k^{cut} \in \{0, 1\}^{|\bar{C}_k|}, \quad (17)$$

表示从候选割池 $\bar{C}_k$ 中选择哪些割优先加入主问题。

状态转移. 给定 s_k 和 a_k , 环境执行一轮 LBBD: 求解主问题, 调用 CP/MIP 子问题, 生成割, 更新 incumbent、上下界和历史统计, 得到 s_{k+1} 。该转移由优化器和排程验证器定义, 不要求可微。

奖励函数. 为降低稀疏奖励带来的训练难度, 本文采用 dense reward:

$$r_k = w_1 (\text{Obj}(x_{k-1}^{best}) - \text{Obj}(x_k^{best})) + w_2 (\text{Gap}_{k-1} - \text{Gap}_k) - w_3 T_k^{SP} - w_4 |R_k| - w_5 V_k, \quad (18)$$

其中 T_k^{SP} 为子问题求解时间, V_k 为不可行惩罚或稳定性违反程度。episode 总回报为

$$R(\tau) = \sum_{k=1}^K \gamma_{\tau k}^{k-1} r_k. \quad (19)$$

4.4 异构图状态编码与策略网络

本文拟将调度状态表示为异构图 $G_k = (V, E)$ 。节点类型包括 operation node、machine node、job node、cut node 和 disturbance node。边类型包括工艺前后边、机器候选边、当前同机序列边、原始同机序列边、critical-path 边、扰动影响边和 cut-involves-operation 边。

操作节点特征包括加工时间、候选机器数、原开始时间、当前开始时间、开始时间偏移、slack、是否在关键路径上、是否受扰动影响、是否曾被修复、due-date pressure 等。机器节点特征包括负载、故障窗口、利用率、受影响操作数量和历史修复收益。割节点特征包括割类型、年龄、是否曾导致下界改善、是否导致子问题可行性改善。

策略网络采用异构消息传递:

$$h_v^{(l+1)} = \sigma \left(W_{\tau(v)} h_v^{(l)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{c_{v,r}} W_r h_u^{(l)} \right), \quad (20)$$

其中 $\tau(v)$ 为节点类型, r 为边类型。得到节点嵌入后, 分别构造锚点选择、邻域类型选择、半径选择和预算选择的策略头:

$$\pi_{\theta}(a^{anchor} = o | s_k) = \text{softmax}_o(\text{MLP}_{anchor}(h_o)), \quad (21)$$

$$\pi_{\theta}(a^{type} | s_k) = \text{softmax}(\text{MLP}_{type}(h_G)), \quad (22)$$

$$\pi_{\theta}(a^{radius} | s_k) = \text{softmax}(\text{MLP}_{radius}(h_G)), \quad (23)$$

$$\pi_{\theta}(a^{budget} | s_k) = \text{softmax}(\text{MLP}_{budget}(h_G)), \quad (24)$$

其中 h_G 为图级 readout 表示。

4.5 训练策略: 模仿预训练与强化学习微调

从零训练 RL 策略可能样本效率低、方差大。本文采用两阶段训练。

模仿预训练. 构造若干 expert 策略, 包括 critical-path-first、largest-delay-first、affected-machine-first、plain-LBBD best-improvement oracle 等。给定求解轨迹数据 $\mathcal{D} = \{(s_i, a_i^*)\}$, 最小化交叉熵损失:

$$\mathcal{L}_{IL}(\theta) = - \sum_i \log \pi_{\theta}(a_i^* | s_i). \quad (25)$$

该阶段使策略获得基本修复能力, 避免 RL 初期频繁选择无效邻域。

强化学习微调. 在预训练基础上, 采用 PPO 或 actor-critic 对策略进行微调:

$$\max_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{k=1}^K \gamma_{rl}^{k-1} r_k \right] - \lambda_{KL} D_{KL}(\pi_{\theta} || \pi_{ref}), \quad (26)$$

其中 KL 正则用于防止策略偏离稳定 expert 过远。对于 cut selection, 可采用 pointer network 或 set transformer 对候选割集合编码, 并用二元选择或排序损失训练。

4.6 算法流程

本文拟实现的核心算法如下。

```

Algorithm: RL-Guided Unrolled LBBD for Dynamic FJSP Repair
Input: original schedule x0, disturbance d, max iterations K, policy pi_theta
Initialise incumbent x_best by quick repair; initialise master MP_0 and cut
      pool C
for k = 1, ..., K do
    Extract graph state s_k from x_best, d, bounds, cuts and history
    Sample action a_k ~ pi_theta(. | s_k)
    Decode a_k into repair region R_k and subproblem budget tau_k
  
```

```

Solve master problem MP_k with current cut pool C
Solve scheduling subproblem SP_k restricted by R_k and tau_k
if SP_k is infeasible then
    Generate feasibility cut c_k
else
    Obtain schedule x_k and update incumbent if Obj(x_k) improves
    Generate optimality / critical-path / stability cut c_k
end if
Add c_k to cut pool C according to cut management policy
Compute reward r_k from objective improvement, gap reduction and runtime
end for
Update pi_theta using imitation learning and RL fine-tuning
Return best repaired schedule x_best

```

4.7 理论性质与可解释性讨论

本文方法的关键理论定位是：RL 改变搜索控制顺序，而不改变底层可行性约束。若策略只影响修复区域排序、割优先级和预算分配，并在时间允许时回退到完整 LBBD 策略，则可保持 LBBD 的完备性恢复能力。形式上，设 $\mathcal{A}_{full}$ 为完整 LBBD 在有限实例上最终会枚举或排除的高层决策集合，RL 策略诱导一个访问顺序 σ_θ 。若满足

$$\{a : \exists t, \sigma_\theta(t) = a\} = \mathcal{A}_{full}, \quad (27)$$

且所有必要逻辑割最终不会被永久删除，则 RL-guided LBBD 与 plain LBBD 在无限预算下具有相同的可行决策覆盖范围。实际 anytime 设置下，本文关注的是在有限预算 B 内最大化解质量：

$$\max_{\pi_\theta} \mathbb{E}_{I \sim \mathcal{P}} [Q(\text{LBBD}_{\pi_\theta}(I, B))], \quad (28)$$

其中 Q 衡量给定时间预算内的目标值、稳定性和可行性。这一定位能够回应审稿人对“学习策略是否破坏精确性”的潜在疑问：本文主算法面向 anytime repair，精确性通过 fallback 完整 LBBD 机制保留，学习模块主要提升有限预算下的搜索效率。

5 实验设计与预期结果（包括数据集、基线方法、评价指标、消融实验、可能的定性分析等）

5.1 数据集与实例生成

实验将覆盖三类实例。

标准静态实例加扰动. 选取经典 JSSP/FJSP 基准实例, 如 Brandimarte、Hurink、Dauzere-Peres、Taillard 等实例集 [17, 18]. 先用 CP-SAT、CP Optimizer 或强启发式求得初始排产, 再注入扰动形成动态重调度任务。

合成动态 FJSP 实例. 按照工件数 $J \in \{10, 20, 50, 100\}$ 、机器数 $M \in \{5, 10, 20\}$ 、操作数、机器柔性度、due date tightness 等参数生成实例。扰动类型包括：

- 机器故障：随机机器在随机时间窗口不可用；
- 新订单到达：在当前排产执行中途插入新工件；
- 加工时间变化：若干操作加工时间延长；
- 急单插入：新工件具有较紧 due date；
- 混合扰动：多个扰动同时出现。

跨分布泛化测试. 训练分布采用中小规模均匀实例, 测试分布扩展到更大规模、更高机器柔性度、更强 due-date pressure、更长故障窗口和不同扰动组合, 以检验方法的泛化能力。

5.2 基线方法

实验将比较以下基线：

1. **Full CP/MIP Rescheduling**: 忽略稳定性或加入稳定性项后从零重排；
2. **Plain LBBD**: 固定规则选择修复区域和割平面；
3. **Critical-Path LBBD**: 优先修复关键路径或关键机器块；
4. **Random-Region LBBD**: 随机选择修复区域, 用于验证学习策略有效性；
5. **Rolling Horizon Repair**: 滚动窗口重调度方法；
6. **Dispatching Rules**: 如 SPT、LPT、MWKR、EDD、CR 等；
7. **Metaheuristics**: ALNS、tabu search、simulated annealing 或遗传算法；
8. **GNN/RL Dispatching**: 图神经网络编码状态并逐步选择操作的学习型调度方法；
9. **Oracle Region Upper Bound**: 用离线搜索选择每轮最佳修复区域, 作为可达上界参考。

5.3 评价指标

评价指标将覆盖质量、稳定性、效率和求解过程四个方面：

- **目标质量**: $C_{\max}$ 、总拖期、加权目标 $\text{Obj}(x)$;
- **稳定性**: 平均开始时间偏移、机器重分配率、同机序列扰动距离;
- **可行性**: 可行率、约束违反数量、恢复可行所需时间;
- **求解效率**: 总运行时间、子问题平均时间、LBBD 迭代次数、割数量;
- **anytime 性能**: 5 秒、10 秒、30 秒、60 秒预算下的 objective-time curve;
- **gap 指标**: 相对最优 gap 或相对最强 baseline gap;
- **泛化指标**: 跨规模、跨扰动类型和跨实例分布的性能下降幅度。

5.4 消融实验

拟设计以下消融实验:

1. **无 RL 控制**: 将策略替换为固定 critical-path 规则, 验证学习控制贡献;
2. **无模仿预训练**: 直接 RL 训练, 验证 imitation warm start 的必要性;
3. **无 gap reward**: 去掉式(18)中的 gap reduction 项, 分析下界信息对学习的影响;
4. **无稳定性特征**: 去掉开始时间偏移、机器重分配等特征, 验证 rescheduling-specific features 的作用;
5. **仅 region selection vs. region + budget selection vs. region + budget + cut selection**, 分析不同控制动作的边际收益;
6. **GNN 编码器替换**: 比较 MLP、同构 GNN、异构 GNN 和 Transformer-style set encoder;
7. **修复半径敏感性**: 分析小邻域、中邻域和大邻域对质量与时间的影响;
8. **fallback 机制**: 比较有无完整 LBBD fallback 对可行率和最坏情形性能的影响。

5.5 定性分析

除定量指标外, 本文将进行以下定性分析:

- 可视化 RL 策略在机器故障后选择的修复区域, 观察其是否沿关键路径、故障传播链或 due-date 紧迫工件展开;
- 分析不同扰动类型下策略对邻域半径和子问题预算的自适应变化;
- 可视化割池演化, 评估 RL 是否优先选择高效 critical-path cuts 或 feasibility cuts;
- 展示若干 case study, 对比 full reschedule 与 repair schedule 的稳定性差异;

- 对失败案例进行归因, 判断是状态表示不足、奖励误导、子问题时间限制还是 LBBD 割不够强。

5.6 预期结果

预期本文方法在有限时间预算下相对于 plain LBBD 和 critical-path LBBD 能取得更好的 anytime objective, 尤其在中大规模动态实例和混合扰动场景中表现更明显。相对于纯 GNN/RL dispatching 方法, 预期本文能获得更高可行率和更低稳定性扰动。相对于 full CP/MIP rescheduling, 预期本文在短时间预算下能更快产生高质量可行解, 并显著降低机器重分配和开始时间偏移。消融实验预期将表明: 异构图表示、模仿预训练、gap-aware reward 和预算控制均对性能有实质贡献。

6 预期贡献与创新点总结

本文预期贡献如下:

1. **问题建模贡献:** 提出带稳定性正则的动态 FJSP 修复建模框架, 将效率、拖期、机器重分配和序列扰动统一进 proximal rescheduling objective。
2. **算法框架贡献:** 构建面向动态重调度的 LBBD 分解框架, 使主问题负责高层修复计划, 子问题负责局部精确排程, 并通过 feasibility、critical-path、stability-aware cuts 迭代反馈。
3. **学习机制贡献:** 首次系统地将 LBBD 迭代过程建模为可展开 MDP, 使强化学习策略学习修复区域、子问题预算和割平面优先级, 而非直接生成完整调度方案。
4. **模型设计贡献:** 设计面向排产状态、扰动事件和割池信息的异构图状态编码器, 使策略能够利用工艺约束、机器负载、关键路径和历史求解轨迹。
5. **实验评估贡献:** 建立覆盖静态基准加扰动、合成动态实例和跨分布泛化测试的系统实验框架, 并从 objective、stability、feasibility、runtime 和 anytime curve 多维评估算法。
6. **范式贡献:** 为组合优化中的 “unroll + RL” 提供一种非黑箱、可解释、可与传统求解器结合的研究路径, 即学习不替代求解器, 而是控制求解器的展开过程。

7 潜在局限与未来工作

从审稿角度看, 本文可能面临以下质疑。

质疑一：RL 是否真正优于人工规则？ 动态调度中 critical-path-first、largest-delay-first 等规则已经较强。若 RL 仅在少数实例上略优，则创新性不足。对此，本文将通过 random-region、critical-path LBBD、plain LBBD 和 oracle-region upper bound 的系统对比，验证 RL 在跨扰动类型和跨规模场景中的稳定收益；同时通过消融分析证明策略学习到的不只是关键路径规则的简单复制。

质疑二：学习策略是否破坏 LBBD 的完备性？ 若 RL 删除必要割或永久跳过某些候选区域，可能破坏精确性。本文将明确区分 anytime learning mode 与 exact fallback mode。RL 只改变搜索优先级、预算分配和候选修复区域顺序；在需要精确保证时，算法可回退到完整 LBBD 枚举与完整割添加机制。由此，学习模块不修改底层约束逻辑，而主要改善有限预算下的效率。

质疑三：子问题调用 CP/MIP 会导致训练成本过高。 这是学习增强优化常见问题。本文将采用三种缓解策略：第一，离线收集 plain LBBD 轨迹进行模仿预训练；第二，对子问题采用分级预算和 warm start；第三，在训练中缓存相似 repair region 的子问题结果，并优先在中小规模实例上训练，再进行跨规模测试。必要时可使用 surrogate evaluator 进行预训练，再用真实求解器微调。

质疑四：方法是否过度工程化？ LBBD、CP-SAT、GNN 和 RL 的组合可能显得复杂。本文将以最小可行版本为主线，即 RL-Region-LBBD：只让 RL 选择修复区域，其余 LBBD 组件保持固定。随后再逐步扩展到预算控制和割选择。这样的分阶段设计能够清晰评估每个模块的贡献，避免“堆模块”式研究。

质疑五：泛化能力是否可靠？ 学习型组合优化方法常在训练分布表现良好，但跨规模或跨分布退化明显。本文将在实验设计中强制包含 train-small/test-large、uniform-to-clustered、single-disturbance-to-mixed-disturbance 等泛化设置，并报告性能下降曲线。状态编码中使用相对特征、图结构特征和局部邻域动作，而非固定维度全局编码，以提高泛化潜力。

未来工作可从四方面展开。第一，进一步研究具有理论保证的 safe RL 控制，使策略在有限预算下也能满足更强的可行性恢复约束。第二，将该框架拓展到多目标生产调度、能源感知调度和人机协作调度。第三，探索大语言模型作为高层算法设计器，用于自动生成修复邻域、逻辑割模板或实验实例。第四，将 LBBD + RL 展开框架推广到车辆路径、机组排班、手术室调度和云计算任务调度等其他组合优化场景。

8 结论

本文提出的博士研究方案聚焦于动态柔性作业车间重调度中的一个关键问题：如何在复杂扰动下快速生成兼顾效率、稳定性和可行性的修复方案。与传统精确算法相比，

本文引入强化学习以改善 LBBD 的搜索控制；与纯学习型调度方法相比，本文保留主问题、子问题和逻辑割所提供的可解释优化结构。核心思想是将 LBBD 的迭代过程视为可展开求解轨迹，使 RL 策略学习每一轮“修哪里、解多久、加哪些信息”，而不是直接学习完整排产方案。该框架有望在理论上丰富学习增强分解优化的研究范式，在方法上形成可验证、可解释、可泛化的 unrolled solver，在应用上提升智能制造系统面对动态扰动时的快速重调度能力。

参考文献

- [1] Placeholder Author. A survey on flexible job-shop scheduling problems and solution methods. *European Journal of Operational Research*, 20XX.
- [2] Placeholder Author. *Scheduling: Theory, Algorithms, and Systems*. Springer, 20XX.
- [3] Placeholder Author. Dynamic scheduling and rescheduling in manufacturing systems: A review. *International Journal of Production Research*, 20XX.
- [4] Placeholder Author. Adaptive large neighbourhood search for scheduling and routing. *Computers & Operations Research*, 20XX.
- [5] Placeholder Author. Logic-based Benders decomposition for planning and scheduling. *INFORMS Journal on Computing*, 20XX.
- [6] Placeholder Author. Constraint programming approaches for flexible job-shop scheduling. *Operations Research*, 20XX.
- [7] Placeholder Author. Interval variables and optional activities in constraint programming for scheduling. *CP Optimizer Documentation*, 20XX.
- [8] Placeholder Author. Job shop scheduling with CP-SAT. *OR-Tools Documentation*, 20XX.
- [9] Placeholder Author. Learning to dispatch for job shop scheduling via deep reinforcement learning and graph neural networks. *NeurIPS / ICLR Workshop*, 20XX.
- [10] Placeholder Author. Neural combinatorial optimization with reinforcement learning. *International Conference on Learning Representations*, 20XX.
- [11] Placeholder Author. Learning combinatorial optimization algorithms over graphs. *NeurIPS*, 20XX.
- [12] Placeholder Author. Attention, learn to solve routing problems. *International Conference on Learning Representations*, 20XX.

- [13] Placeholder Author. Algorithm unrolling: Interpretable, efficient deep learning for optimization. *IEEE Signal Processing Magazine*, 20XX.
- [14] Placeholder Author. Learning to branch in mixed integer programming. *International Conference on Machine Learning*, 20XX.
- [15] Placeholder Author. Learning to cut: Machine learning for Benders decomposition and cutting-plane selection. *NeurIPS / ICML*, 20XX.
- [16] Placeholder Author. Learning large neighbourhood search for combinatorial optimization. *International Conference on Learning Representations*, 20XX.
- [17] Placeholder Author. Benchmark instances for flexible job-shop scheduling. *OR Library / Scheduling Benchmarks*, 20XX.
- [18] Placeholder Author. Taillard and Brandimarte benchmark instances for job-shop and flexible job-shop scheduling. *Benchmark Collection*, 20XX.